



Ain Shams University – Faculty of Engineering I-CHEP
CSE356: Internet of Things

Project Report

Smart Mosque Management System

14-May-2026

Submitted to	Prof. Islam Tharwat
	Eng. Hamdy Osama

Submitted by	23P0184 – Sara Ahmed
	23P0360 – El-Sharqawy Wael
	23P0386 – Esraa Mustafa
	23P0272 – Nada Medhat

Contents

1	Purpose & Requirements	4
1.1	Purpose of the IoT System	4
1.2	Functional Requirements	4
1.3	Non-Functional Requirements	5
1.4	Constraints and Assumptions	6
2	Process Specification	6
2.1	Overview	6
2.2	Core System Processes	6
2.2.1	Occupancy Detection Process	6
2.2.2	Energy Management Process	7
2.2.3	Wudu Water Monitoring Process	7
2.2.4	Air Quality Monitoring Process	7
2.2.5	Central Monitoring and Control Process	8
2.3	Integrated System Flowchart	8
2.4	Process Interaction Notes	8
3	Domain Model Specification	9
3.1	Entities	9
3.2	Attributes for each Entity	9
3.3	Relationships	11
3.4	UML Diagram	11
4	Domain Model Specification	11
4.1	Entities	11
4.2	Attributes for each Entity	12
4.3	Relationships	14
4.4	UML Diagram	14
5	Information Model Specification	14
5.1	Types of Data in System	14
5.1.1	Sensor Data (Raw Data)	14
5.1.2	Processed Data	15
5.1.3	System Data (Stored)	15
5.1.4	Control Data (from Admin)	15
5.2	Data Flow	15
5.2.1	Main Flow	15
5.2.2	Control Flow	16
6	Service Specifications	16
6.1	Overview	16
6.2	Service List	17
6.3	Service Descriptions	17
7	Service Specifications	20
7.1	Overview	20
7.2	Service List	20
7.3	Service Descriptions	20
8	IoT Level Specification	24
8.1	Level 1 – Perception Layer	25

8.2	Level 2 – Network Layer	25
8.3	Level 3 – Edge Processing Layer	26
8.4	Level 4 – Data Accumulation Layer	26
8.5	Level 5 – Application Layer	26
8.6	Level Interaction Summary	27
9	Functional View Specification	27
9.1	Overview	27
9.2	Functional Block Descriptions	27
9.3	Inter-Block Data Flow Summary	29
10	Operational View Specification	29
10.1	System Performance	29
10.1.1	Response Time	29
10.1.2	Data Update Frequency	30
10.1.3	Throughput	30
10.1.4	Efficiency	30
10.2	System Reliability	31
10.2.1	Sensor Failure Handling	31
10.2.2	Network Failure	31
10.2.3	Power Failure	31
10.3	System Scalability	31
10.3.1	More Sensors	31
10.3.2	Spreading to Other Mosques	31
10.3.3	Modular Architecture	32
10.4	System Security	32
10.4.1	Network Protection	32
10.4.2	Access Control	32
10.4.3	Data Encryption	32
10.4.4	Device Filtering	33
10.4.5	Privacy Protection	33
10.5	Energy Efficiency	33
10.5.1	Prayer Time Control	33
10.5.2	Occupancy-Based Activation	33
10.5.3	Automatic Shutdown	33
10.6	Safety Considerations	33
10.6.1	Device Overheating	34
10.6.2	Water Valve Reliability	34
11	Device and Component Integration	34
11.1	Input Devices (Sensors)	34
11.1.1	Motion Sensor (PIR)	34
11.1.2	Air Quality Sensor (IAQ)	36
11.1.3	Water Flow Sensor	37
11.2	Output Devices (Actuators)	38
11.2.1	Smart Lights	38
11.2.2	Fans (AC Simulation)	39
11.2.3	Solenoid Valve	39
11.2.4	Smart Window	39
11.2.5	Humidifier	40
11.2.6	Street Lamp	40
11.3	Processing Unit	41
11.4	Communication Layer	41

11.5	Integration Workflow	41
11.6	System Diagram	42
12	Application Development	42
12.1	Application Overview and Platform	42
12.2	User Role	43
12.3	Core Features	43
12.4	User Interface Design	43
12.5	System Integration with the Dashboard	45
	List of Figures	46
	References	47

1. Purpose & Requirements

1.1 Purpose of the IoT System

The purpose of the Smart Mosque Management System is to design an intelligent IoT-based solution that enhances the operational efficiency of mosques by:

- Optimizing energy consumption (lighting and air conditioning)
- Conserving water usage in wudu (ablution) areas
- Monitoring and managing crowd occupancy
- Improving indoor air quality and comfort

The system leverages real-time sensor data, automated control mechanisms, and a centralized monitoring dashboard to ensure efficient resource utilization, especially during high-attendance periods such as Friday prayers and Ramadan.

1.2 Functional Requirements

These define what the system must do.

A. Energy Management

- Automatically turn ON/OFF lights and fans/AC based on:
 - Prayer times (via API)
 - Occupancy detection (motion sensors)
- Adjust energy usage dynamically depending on crowd density.
- Smart window control: automatically opens for natural ventilation when AC is off and humidity $\geq 60\%$; closes when AC activates to retain conditioned air.
- Humidity comfort control: humidifier activates when indoor humidity drops below 30%.

B. Occupancy Detection

- Count number of people entering/exiting using PIR sensors.
- Provide real-time hall capacity status.
- Activate a dedicated siren (is-mosque-full-siren) when all three prayer hall PIR sensors are simultaneously active, indicating maximum occupancy.

C. Water Monitoring (Wudu Area)

- Measure water flow using flow sensors.
- Detect abnormal usage (leaks or taps left open).
- Automatically close water supply using water valves.

D. Air Quality Monitoring

- Measure air quality (e.g., CO₂, humidity, temperature).
- Trigger ventilation (fans) when thresholds are exceeded.

E. Central Dashboard

- Display:
 - Current occupancy
 - Energy consumption
 - Water usage
 - Air quality readings
- Allow manual override control for administrators.

F. Notifications & Alerts

- Send alerts for overcrowding, water leakage, poor air quality, and system faults.

1.3 Non-Functional Requirements

These define how well the system should perform.

A. Performance

- Real-time response (low latency in sensor-actuator loop)
- Efficient handling of multiple sensors simultaneously

B. Scalability

- System should support small to large mosques
- Easy addition of new sensors/devices

C. Reliability

- Continuous operation with minimal downtime
- Fault-tolerant communication between devices

D. Security

- Encrypted communication using WPA2-PSK for wireless sensor-to-gateway connections
- Dashboard access control with username/password authentication
- MAC filtering to allow only authorized IoT nodes

E. Usability

- Simple and intuitive dashboard interface

- Minimal training required for mosque staff

F. Maintainability

- Easy software/hardware updates
- Modular design for replacing faulty components

G. Cost Efficiency

- Use affordable IoT components suitable for local deployment in Egypt

1.4 Constraints and Assumptions

Constraints

- Limited budget; use cost-effective sensors
- Implementation in Cisco Packet Tracer simulation
- Dependence on internet for prayer-time API

Assumptions

- Stable local network exists in mosque
- Power supply is reliable
- Admin has basic technical knowledge

The system aligns with Egypt's sustainability goals by reducing unnecessary consumption of water and electricity in high-usage public religious spaces.

2. Process Specification

2.1 Overview

The Smart Mosque Management System operates through coordinated processes that continuously collect sensor data, analyze conditions, and trigger automated actions. These processes ensure efficient energy usage, water conservation, and safe occupancy management. The system follows a Sense → Process → Act → Monitor pipeline.

2.2 Core System Processes

2.2.1 Occupancy Detection Process

Objective: Monitor the number of people inside the mosque.

Workflow:

1. PIR sensors at entrances detect entry/exit events.
2. System increments/decrements occupancy count.

3. Data is sent to the central controller.
4. Trigger occupancy-full siren (is-mosque-full-siren) when all hall motion detectors activate simultaneously, and deactivate when any clears.
5. Occupancy data is used by other processes (energy control).

2.2.2 Energy Management Process

Objective: Optimize electricity usage based on prayer times and occupancy.

Workflow:

1. System retrieves prayer times from API.
2. Motion sensors detect presence in prayer hall.
3. Decision logic:
 - IF (Within Prayer Window OR Motion Detected), then turn ON lights and AC/fans.
 - ELSE, turn OFF devices.
4. System also activates AC independently when temperature exceeds 32°C (separate threshold from fans at 28°C). When AC activates, smart window automatically closes. When AC is off and humidity $\geq 60\%$, window opens for passive ventilation.
5. Optionally adjust intensity based on crowd density.

2.2.3 Wudu Water Monitoring Process

Objective: Prevent water waste in ablution areas.

Workflow:

1. Flow sensors measure water usage.
2. System checks for continuous flow without interruption.
3. If abnormal usage is detected:
 - Assume tap left open or leak.
 - Activate water valve to shut water.
4. Log water consumption data.

2.2.4 Air Quality Monitoring Process

Objective: Maintain comfortable and safe air conditions.

Workflow:

1. Sensors read CO₂, humidity, and temperature.
2. Compare readings with predefined thresholds.
3. If air quality is poor, activate ventilation.
4. Humidifier activates when humidity falls below 30% to maintain comfort.

5. Window management provides passive ventilation as a complement to mechanical AC.
6. Update dashboard with real-time values.

2.2.5 Central Monitoring and Control Process

Objective: Provide full system visibility and control.

Workflow:

1. Collect data from all subsystems.
2. Display occupancy, energy usage, water consumption, and air quality on web dashboard.
3. Allow admin override and manual ON/OFF operations.
4. Optionally store historical data.

2.3 Integrated System Flowchart

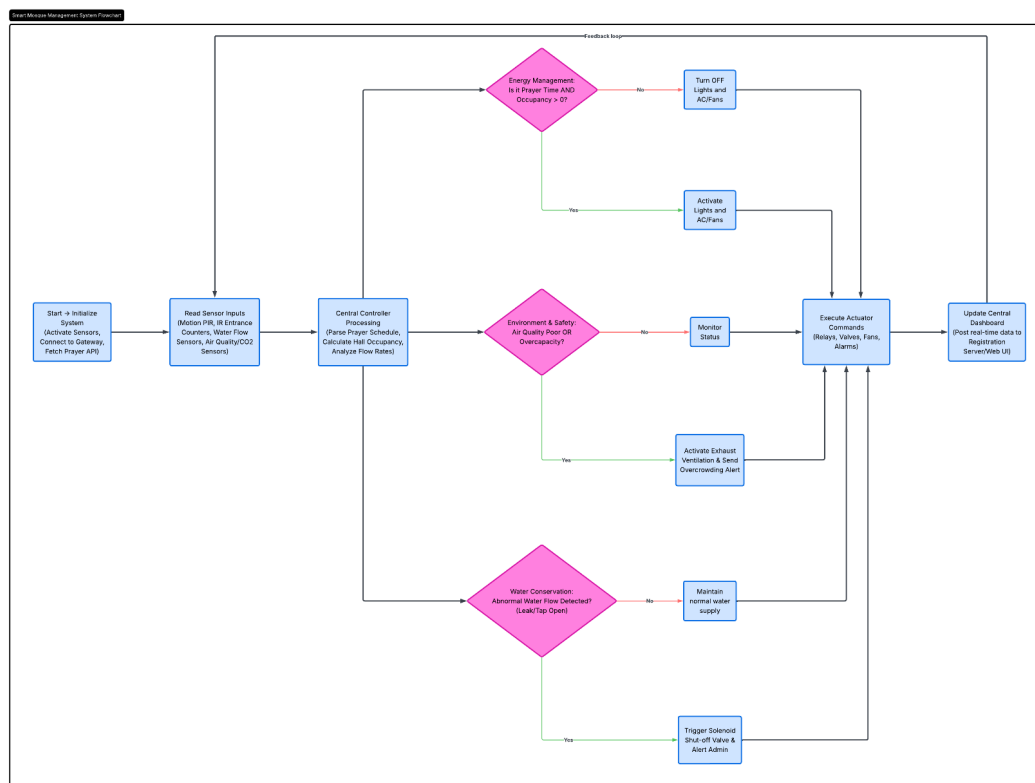


Figure 1: Integrated system flowchart

2.4 Process Interaction Notes

- Occupancy data affects energy decisions.
- Air quality can override energy logic (force ventilation ON).
- Water system is independent but logged centrally.
- All processes communicate via a central controller (MCU/SBC).

The system processes are designed as loosely coupled modules to ensure scalability, maintainability, and real-time responsiveness. «««< HEAD

3. Domain Model Specification

3.1 Entities

Core System Entities:

- Mosque
- Zone (Prayer Hall / Wudu Area)
- Smart Window
- Humidifier
- Street Lamp
- Occupancy Siren

Sensors:

- Motion Sensor
- Air Quality Sensor
- Water Flow Sensor

Actuators:

- Light
- Fan / AC
- Water Valve

System Components:

- Controller (MCU/SBC)
- IoT Server
- Dashboard (Web App)
- Admin

3.2 Attributes for each Entity

Mosque

- mosqueID
- name
- location

Zone

- zoneID
- type (Prayer Hall / Wudu Area)
- capacity

Sensor

- sensorID
- type (motion, PIR, air, water)
- status (active/inactive)
- value (reading)

Actuator

- actuatorID
- type (light, fan, valve)
- status (ON/OFF)

Controller

- controllerID
- processingLogic
- status

IoT Server

- serverID
- database
- connectionStatus

Dashboard

- dashboardID
- displayData
- controlOptions

Admin

- adminID
- username
- password

3.3 Relationships

- Mosque contains Zones
- Zone contains Sensors
- Zone contains Actuators
- Sensors send data to Controller
- Controller controls Actuators
- Controller communicates with IoT Server
- IoT Server updates Dashboard
- Admin interacts with Dashboard
- Dashboard sends commands to Controller

3.4 UML Diagram

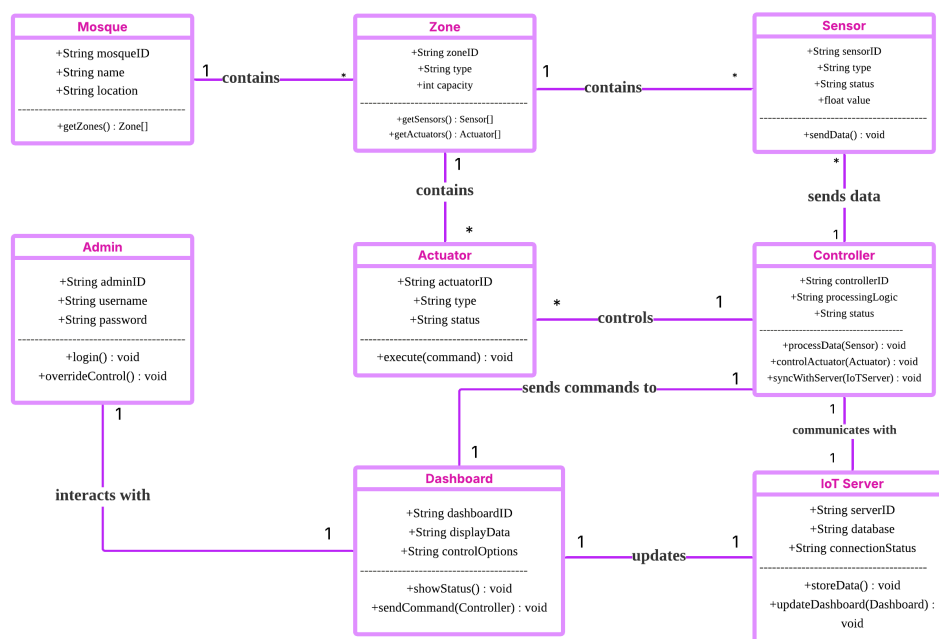


Figure 2: UML domain model diagram

=====

4. Domain Model Specification

4.1 Entities

Core System Entities:

- Mosque

- Zone (Prayer Hall / Wudu Area)

Sensors:

- Motion Sensor
- IR Counter Sensor
- Air Quality Sensor
- Water Flow Sensor

Actuators:

- Light
- Fan / AC
- Water Valve

System Components:

- Controller (MCU/SBC)
- IoT Server
- Dashboard (Web App)
- Admin

4.2 Attributes for each Entity

Mosque

- mosqueID
- name
- location

Zone

- zoneID
- type (Prayer Hall / Wudu Area)
- capacity

Sensor

- sensorID
- type (motion, PIR, air, water)
- status (active/inactive)
- value (reading)

Actuator

- actuatorID
- type (light, fan, valve)
- status (ON/OFF)

Controller

- controllerID
- processingLogic
- status

IoT Server

- serverID
- database
- connectionStatus

Dashboard

- dashboardID
- displayData
- controlOptions

Admin

- adminID
- username
- password

4.3 Relationships

- Mosque contains Zones
- Zone contains Sensors
- Zone contains Actuators
- Sensors send data to Controller
- Controller controls Actuators
- Controller communicates with IoT Server
- IoT Server updates Dashboard
- Admin interacts with Dashboard
- Dashboard sends commands to Controller

4.4 UML Diagram

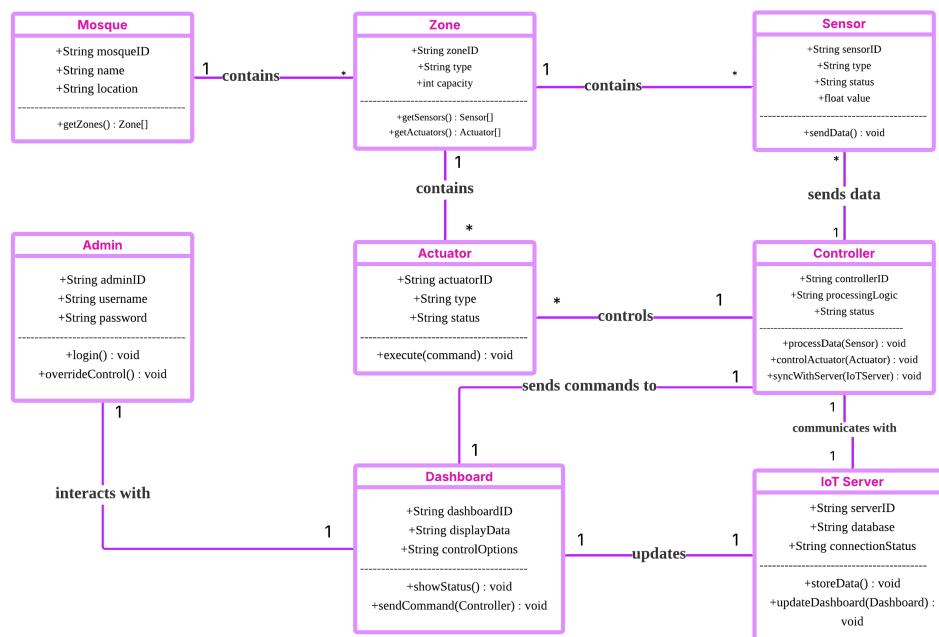


Figure 3: UML domain model diagram

» » » > e43e52f20d321dd86cf0dd1f5e670263b6aa2823

5. Information Model Specification

5.1 Types of Data in System

5.1.1 Sensor Data (Raw Data)

Occupancy Data

- peopleCount (int)
- entry/exit events

Motion Data

- motionDetected (boolean)

Air Quality Data

- CO₂Level (ppm)
- temperature (°C)
- humidity (%)

Water Data

- flowRate (L/min)
- usageDuration (seconds)

5.1.2 Processed Data

- occupancyStatus (normal / full / overcrowded)
- airQualityStatus (good / moderate / poor)
- waterUsageStatus (normal / leak detected)
- energyState (ON / OFF)

5.1.3 System Data (Stored)

- historical water usage
- energy consumption logs
- occupancy logs
- alerts history

5.1.4 Control Data (from Admin)

- manual override commands
- turn ON/OFF devices

5.2 Data Flow

5.2.1 Main Flow

Sensors → Controller → IoT Server → Dashboard

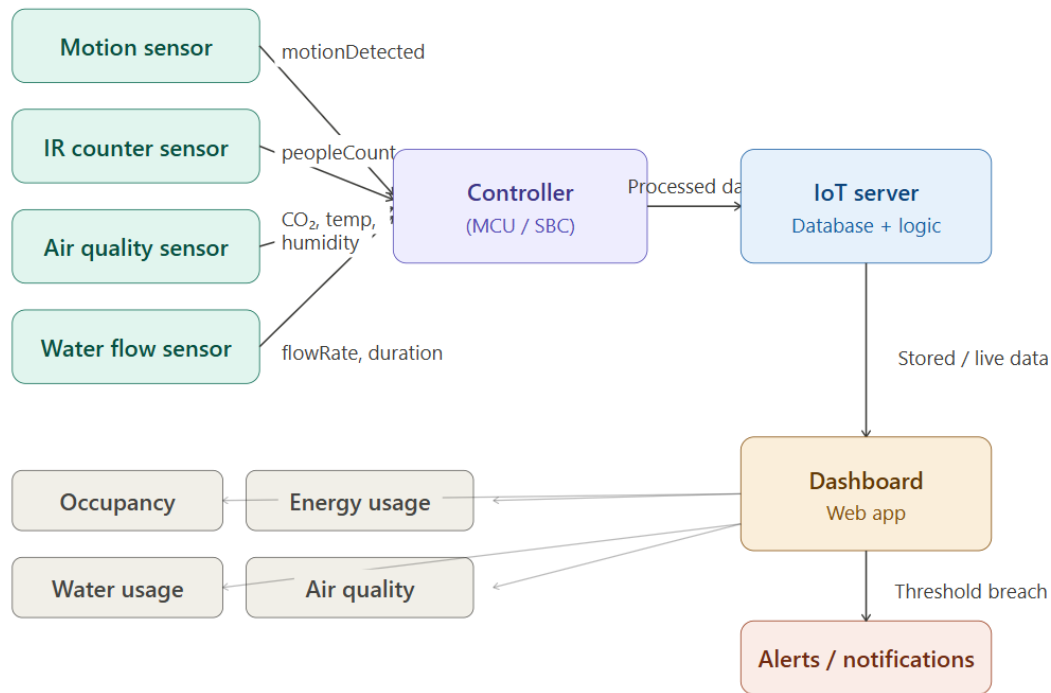


Figure 4: Main data flow

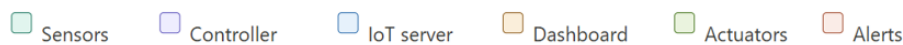


Figure 5: Main flow legend

5.2.2 Control Flow

Admin → Dashboard → Controller → Actuators

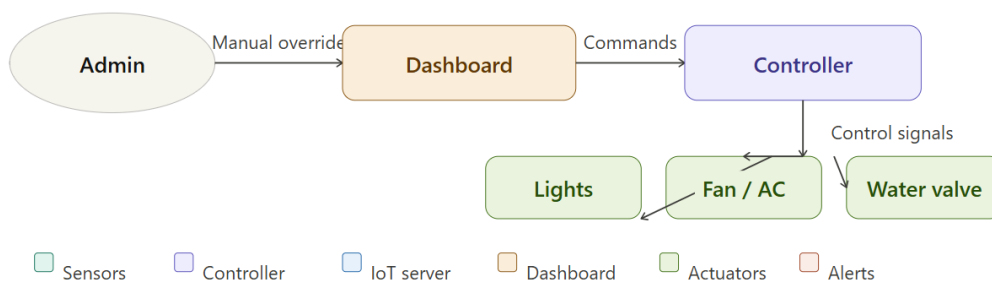


Figure 6: Control flow

« « « < HEAD

6. Service Specifications

6.1 Overview

The Smart Mosque Management System exposes seven core services. Each service is a defined unit of functionality with clear inputs, outputs, triggers, and interfaces. Services are implemented on the

ESP32 controller and exposed to the dashboard via the local network.

6.2 Service List

Service ID	Service Name	Type	Trigger
SVC-01	Prayer Time Sync	Data Acquisition	Scheduled – once daily
SVC-02	Occupancy Tracking	Sensing	Continuous (event-driven)
SVC-03	Energy Control	Actuation	Event-driven
SVC-04	Water Monitoring & Control	Sensing + Actuation	Continuous
SVC-05	Air Quality Monitoring	Sensing + Actuation	Periodic (every 60s)
SVC-06	Alert & Notification	Notification	Threshold-triggered
SVC-07	Dashboard Monitoring & Override	Application + Control	On-demand
SVC-08	Window & Ventilation Control	Actuation	Event-driven (AC state + humidity)
SVC-09	Humidity Comfort Control	Sensing + Actuation	Continuous

Table 1: Core service list

6.3 Service Descriptions

SVC-01 – Prayer Time Sync Service

- Purpose: Retrieves accurate daily prayer times to anchor energy automation logic.
- Interface: Daily HTTP GET request to prayer-time API.
- Output: Five daily timestamps (Fajr, Dhuhr, Asr, Maghrib, Isha) stored locally.
- Fallback: Use previous day’s cached schedule when API is unreachable.

SVC-02 – Occupancy Tracking Service

- Purpose: Counts people in prayer hall in real time.
- Interface: PIR events → ESP32 counter logic → dashboard polling every 30s.
- Operations: `increment_count()`, `decrement_count()`, `get_current_count()`.
- Output fields: `occupancy_count`, `capacity_percent`, `status` (NORMAL / HIGH / FULL).

SVC-03 – Energy Control Service

- Purpose: Automates lights and fans/AC per zone based on prayer windows and occupancy.

- Interface: ESP32 reads SVC-01 and SVC-02 state, then controls relay GPIOs.
- Decision logic:

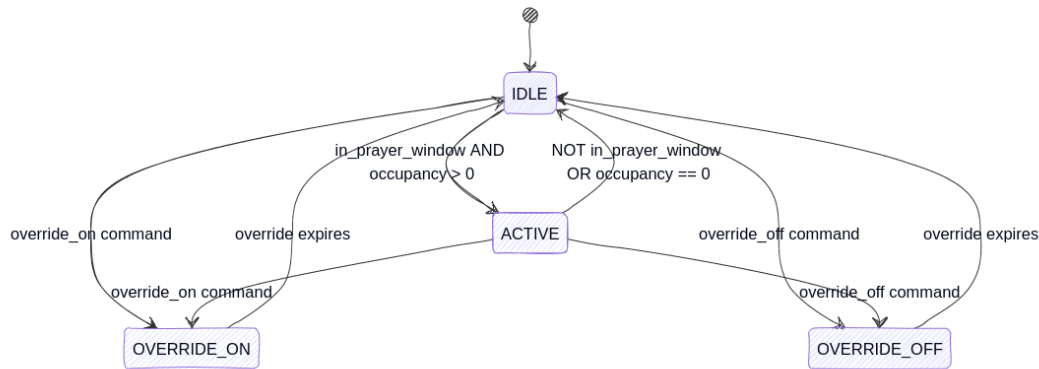


Figure 7: Energy control logic finite state machine

- Air quality from SVC-05 can independently force fan activation.

SVC-04 – Water Monitoring and Control Service

- Purpose: Detect leaks/taps-left-open and control solenoid valve.
- Interface: Flow sensor pulse output → ESP32 pulse counter → valve GPIO.
- Key operations: `monitor_flow()`, `detect_anomaly()`, `close_valve()`, `open_valve()`.
- Failsafe: If flow sensor goes offline, valve defaults to closed and admin is notified.

SVC-05 – Air Quality Monitoring Service

- Purpose: Monitor CO₂, temperature, humidity and auto-activate ventilation.
- Polling interval: Every 60 seconds.

Parameter	Normal	Action Threshold	Action
CO ₂	< 1000 ppm	> 1000 ppm	Activate fans
Temperature	18–26 °C	> 28 °C	Activate fans
Humidity	30–60%	> 65%	Activate fans

Table 2: Air quality thresholds

SVC-06 – Alert and Notification Service

- Purpose: Detect threshold breaches and push real-time alerts.
- Interface: Internal event bus, dashboard polls alert queue.

- Deduplication: Same alert type per device/zone can fire once per 5 minutes.

SVC-07 – Dashboard Monitoring and Manual Override Service

- Purpose: Unified monitoring and manual override control.
- Interface: Dashboard polls ESP32 REST endpoints every 30 seconds; override uses HTTP POST.
- Override operations: `override_on()`, `override_off()`, `clear_override()`.
- Safety: Override expires automatically after configured duration.

SVC-08 – Window and Ventilation Control Service

- Purpose: Control smart window behavior to improve ventilation efficiency and reduce unnecessary cooling loss.
- Interface: IoT Server evaluates humidity sensor readings and AC device status, then sends control actions to the smart window device through the wireless access point.
- Decision logic:
 - IF prayer-AC is ON, set prayer-window state to OFF (closed).
 - IF prayer-AC is OFF AND humidity $\geq 60\%$, set prayer-window state to ON (open).
- Trigger source: Event-driven automation rules configured in the IoT Server Conditions page.
- Connected devices: Humidity Monitor, Air Conditioner, Smart Window.

SVC-09 – Humidity Comfort Control Service

- Purpose: Maintain comfortable indoor humidity levels for worshippers.
- Interface: IoT Server continuously monitors humidity readings and controls the humidifier device through configured automation conditions.
- Decision logic:
 - IF humidity $< 30\%$, activate prayer-humidifier.
 - IF humidity $\geq 30\%$, deactivate prayer-humidifier.
- Trigger source: Continuous threshold monitoring configured in the IoT Server.
- Connected devices: Humidity Monitor and Humidifier.

=====

7. Service Specifications

7.1 Overview

The Smart Mosque Management System exposes seven core services. Each service is a defined unit of functionality with clear inputs, outputs, triggers, and interfaces. Services are implemented on the ESP32 controller and exposed to the dashboard via the local network.

7.2 Service List

Service ID	Service Name	Type	Trigger
SVC-01	Prayer Time Sync	Data Acquisition	Scheduled – once daily
SVC-02	Occupancy Tracking	Sensing	Continuous (event-driven)
SVC-03	Energy Control	Actuation	Event-driven
SVC-04	Water Monitoring & Control	Sensing + Actuation	Continuous
SVC-05	Air Quality Monitoring	Sensing + Actuation	Periodic (every 60s)
SVC-06	Alert & Notification	Notification	Threshold-triggered
SVC-07	Dashboard Monitoring & Override	Application + Control	On-demand

Table 3: Core service list

7.3 Service Descriptions

SVC-01 – Prayer Time Sync Service

- Purpose: Retrieves accurate daily prayer times to anchor energy automation logic. Without this, the system cannot distinguish active prayer windows from idle periods.
- Interface: HTTP GET request to a prayer time API (e.g., aladhan.com/v1/timingsByCity?city=Cairo&count=1) once per day at midnight.
- Output: Five daily timestamps (Fajr, Dhuhr, Asr, Maghrib, Isha) stored in ESP32 local memory.
- Fallback: If the API is unreachable, the system uses previous day's cached schedule. The drift between consecutive days for Egyptian cities is under 2 minutes – acceptable for this use case.

Data produced:

```
{
  "fajr": "04:12",
  "dhuhr": "12:01",
  "asr": "15:28",
  "maghrib": "18:44",
  "isha": "20:10"
}
```

```
}

```

SVC-02 – Occupancy Tracking Service

- Purpose: Counts the number of people inside the prayer hall in real time to support energy decisions and crowd management.
- Interface: PIR sensor events at entry and exit points → ESP32 counter logic → count exposed via local API endpoint polled by dashboard every 30 seconds.
- Operations:
 - `increment_count()` – triggered on entry detection
 - `decrement_count()` – triggered on exit detection; floor is 0 (never negative)
 - `get_current_count()` – returns current integer count and status flag
- Output fields:

Field	Type	Description
occupancy_count	int	Current number of people in hall
capacity_percent	float	$\text{Count} \div \text{max capacity} \times 100$
status	enum	NORMAL / HIGH (>80%) / FULL (=max)

Table 4: Occupancy service output fields

Alert integration: When status transitions to HIGH or FULL, SVC-06 is triggered automatically.

SVC-03 – Energy Control Service

- Purpose: Automatically activates and deactivates lights and fans/AC per zone based on prayer window status and occupancy presence.
- Interface: ESP32 reads state from SVC-01 and SVC-02, applies decision logic, and sends GPIO signals to relay modules controlling lights and fans.
- Decision logic:
 - IF (`current_time` is within `prayer_window`) AND (`occupancy` > 0): `activate(lights, fans)` for active zone
 - ELSE: `deactivate(lights, fans)` for active zone
- IF (`override_active`): ignore above → apply override command from SVC-07
- Operations:

Note: Air quality readings from SVC-05 can override the fan state independently – poor air quality activates fans regardless of prayer window status.

Operation	Description
activate_zone(zone_id, device_type)	Turns on lights or fans in a specific zone
deactivate_zone(zone_id, device_type)	Turns off lights or fans in a specific zone
get_device_status(device_id)	Returns current ON/OFF state of a device

Table 5: Energy control operations

SVC-04 – Water Monitoring and Control Service

- Purpose: Monitors water flow in wudu area, detects leaks or taps left open, and automatically closes solenoid valve after defined inactivity timeout.
- Interface: Water flow sensor (pulse output) → ESP32 pulse counter → solenoid valve GPIO control.
- Key operations:

Operation	Description
monitor_flow(tap_id)	Continuously reads flow rate in L/min
detect_anomaly()	Flags non-zero flow after 30s of no motion nearby
close_valve(tap_id)	Sends close signal to solenoid valve
open_valve(tap_id)	Reopens valve (manual or after reset)
get_water_usage()	Returns cumulative usage in liters per session/day

Table 6: Water monitoring operations

Key thresholds:

Condition	Threshold	Action
Normal use	1–3 L/min	No action
Inactivity timeout	Flow > 0 with no motion for > 30s	Close valve + alert
Leak flag	Flow above normal with no motion	Close valve + high-priority alert

Table 7: Water monitoring thresholds

Failsafe: If flow sensor goes offline, valve defaults to closed and admin is notified.

SVC-05 – Air Quality Monitoring Service

- Purpose: Monitor indoor CO₂ levels, temperature, and humidity. Activates ventilation fans when readings exceed safe thresholds, independent of prayer time logic.
- Interface: IAQ sensor (I²C/UART) → ESP32 → fan relay GPIO + dashboard update.

- Polling interval: Every 60 seconds. Suitable for CO₂ and climate parameters, which change gradually rather than instantaneously.

Output pushed to dashboard every 60s:

```
{
  "co2_ppm": 870,
  "temperature_c": 27,
  "humidity_percent": 58,
  "air_quality_status": "MODERATE"
}
```

Status scale: GOOD (0–50 AQI) / MODERATE (51–100) / POOR (101–150) / UNHEALTHY (151+)

Parameter	Normal	Action Threshold	Action
CO ₂	< 1000 ppm	> 1000 ppm	Activate fans
Temperature	18–26 °C	> 28 °C	Activate fans
Humidity	30–60%	> 65%	Activate fans

Table 8: Air quality thresholds

SVC-06 – Alert and Notification Service

- Purpose: Detects threshold breaches across all subsystems and pushes real-time alerts to admin dashboard.
- Interface: Internal event bus – ESP32 publishes events; dashboard polls the alert queue.
- Deduplication: Same alert type for same device/zone fires at most once per 5 minutes.

Alert definitions:

Alert ID	Trigger Condition	Severity
ALT-01	Occupancy > 80% of max capacity	WARNING
ALT-02	Occupancy = max capacity	CRITICAL
ALT-03	Water flow with no motion > 30s	WARNING
ALT-04	Leak pattern detected (high flow, no motion)	CRITICAL
ALT-05	Air quality status = POOR or UNHEALTHY	WARNING
ALT-06	Sensor offline / no response	FAULT
ALT-07	Actuator failed to respond after 3 retries	FAULT

Table 9: Alert definitions

Alert record stored:


```
{
  "alert_id": "ALT-03",
  "timestamp": "2026-04-25T13:42:00",
  "zone": "wudu_area",
  "status": "ACTIVE"
}
```

SVC-07 – Dashboard Monitoring and Manual Override Service

- Purpose: Provides the mosque administrator with a unified view of all system state and the ability to manually override any automated decision.
- Interface: Web dashboard (hosted on local server/CPT Registration Server) polling ESP32 REST endpoints every 30 seconds. Override commands sent via HTTP POST.

Data displayed:

Panel	Data shown
Occupancy	Live count, capacity bar, status flag
Energy	Device ON/OFF state per zone, prayer schedule
Water	Flow rate per tap, daily usage total, valve state
Air Quality	CO ₂ , temperature, humidity, AQI status
Alerts	Active alert queue with timestamps

Table 10: Dashboard data panels

Override operations:

Operation	Description
override_on(device_id, duration_min)	Forces device ON for N minutes
override_off(device_id)	Forces device OFF regardless of logic
clear_override(device_id)	Returns device to automatic mode

Table 11: Override operations

Safety constraint: Override automatically expires after the set duration. Indefinite override requires admin re-confirmation every 60 minutes.

Use case: Ramadan Tarawih prayers, Eid prayers, and other non-standard events where standard prayer-time logic is insufficient. »»»> e43e52f20d321dd86cf0dd1f5e670263b6aa2823

8. IoT Level Specification

The Smart Mosque Management System is structured across five IoT levels, following a standard layered architecture. Each layer has a defined responsibility and communicates with adjacent layers.

8.1 Level 1 – Perception Layer

Role: Direct interaction with the physical environment.

Sensors

Device	Placement	Measurement
PIR Motion Sensor	Main entrance, exit, hall zone	Entry/exit events, zone presence
Water Flow Sensor	Wudu tap inlet pipe	Flow rate (L/min), pulse output
PIR Motion Sensor (Wudu)	Ablution area	Presence near taps
IAQ Sensor	Prayer hall near ventilation	CO ₂ , temperature, humidity
Humidity Sensor	Prayer hall	Humidity % reading

Table 12: Perception layer sensors

Actuators

Device	Placement	Controlled Action
Smart Relay (Lights)	Per hall zone	ON/OFF lighting
Smart Relay (Fans/AC)	Per hall zone, wudu area	ON/OFF ventilation
Solenoid Valve	Wudu tap supply line	Open/close water flow
Smart Window	Prayer hall	Open/close for ventilation
Humidifier	Prayer hall	ON/OFF for humidity comfort
Street Lamp	Mosque entrance	ON/OFF occupancy-based

Table 13: Perception layer actuators

8.2 Level 2 – Network Layer

Role: Local connectivity and data transport.

- Hardware: Local Wi-Fi access point for ESP32 nodes and local server
- Protocol: IEEE 802.11 Wi-Fi with WPA2-PSK
- Topology: Star
- Segmentation: IoT network isolated from guest/user devices

Key decision: Core control loops are local and do not depend on internet connectivity.

8.3 Level 3 – Edge Processing Layer

Role: Real-time control and decision-making.

Hardware: ESP32 microcontrollers per functional zone.

Function	Description
Sensor polling	Reads PIR, IAQ, and flow sensor inputs at configured intervals
Occupancy counting	Increments/decrements people count based on entry/exit events
Prayer window evaluation	Compares current time against cached prayer schedule
Energy control logic	Combines occupancy and prayer window state for relays
Water control logic	Detects anomalies and closes valve when required
Air quality control	Polls IAQ every 60s and activates fans on threshold breach
Alert generation	Publishes threshold breach events to data layer

Table 14: Edge processing responsibilities

Failsafe: If server connectivity is lost, edge controllers continue autonomous operation.

8.4 Level 4 – Data Accumulation Layer

Role: Aggregate processed data, store logs, and serve dashboard queries.

- CPT simulation backend: Registration Server
- Real deployment option: local server (e.g., lightweight REST + database)
- Retention: rolling 30-day window with optional archival

8.5 Level 5 – Application Layer

Role: Human-facing dashboard for monitoring and control.

- Web dashboard on local network
- Single admin role
- Functions: live monitoring, alert display, schedule view, manual override

8.6 Level Interaction Summary

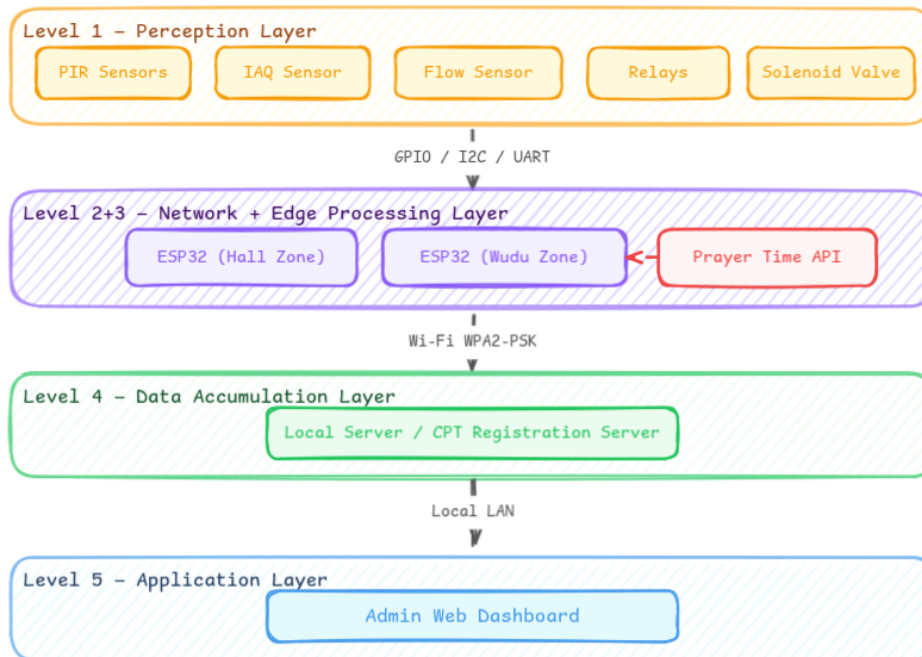


Figure 8: IoT system layer interactions

9. Functional View Specification

9.1 Overview

The functional view breaks the system into independent blocks by behavior. Each block has defined inputs, outputs, and responsibilities, with clear data flows for modularity and maintainability.

9.2 Functional Block Descriptions

Block 1 – Sensing Block

Collects raw data from the mosque environment.

Failure behavior: Offline sensors are flagged to processing, then safe defaults are applied.

Block 2 – Processing Block

Decision core running on ESP32.

- Occupancy counter with floor at zero.
- Prayer window evaluator from cached schedule.
- Energy decision engine for lights/fans.
- Air quality decision engine independent from prayer window.
- Water anomaly detection and valve control.

Sub-function	Sensor	Interval	Output
Entry/exit detection	PIR (entrance)	Event-based	Increment/decrement event
Zone presence detection	PIR (hall zones)	Every 5s	Presence flag per zone
Water flow measurement	Flow sensor	Every 1s	Flow rate (L/min)
Tap-area presence	PIR (wudu)	Event-based	Presence flag
Air quality reading	IAQ sensor	Every 60s	CO ₂ , temperature, humidity

Table 15: Sensing block functions

- State manager as single source of truth.

Block 3 – Actuation Block

Executes commands on relays and valve, with confirmation and retries.

Device	Command Type	Confirmation
Smart relay (lights)	ON/OFF per zone	GPIO acknowledgment
Smart relay (fans/AC)	ON/OFF per zone	GPIO acknowledgment
Solenoid valve	OPEN/CLOSE per tap	GPIO acknowledgment

Table 16: Actuation block control and feedback

Block 4 – Data and Storage Block

Persists time-series data for dashboard and analytics.

Data Type	Logging Frequency	Retention
Occupancy count	Every 30s	30-day rolling
Water usage per tap	Per session + hourly total	30-day rolling
Device state changes	On every transition	30-day rolling
Air quality readings	Every 60s	30-day rolling
Alert records	On every alert event	30-day rolling

Table 17: Stored operational data

Block 5 – Alert Block

Evaluates alert conditions, deduplicates repeated alerts, and pushes active alerts to dashboard.

Block 6 – Dashboard Block

Provides monitoring panels and manual override controls for admin users.

Block 7 – External API Block

Fetches daily prayer times and caches them for resilience.

9.3 Inter-Block Data Flow Summary

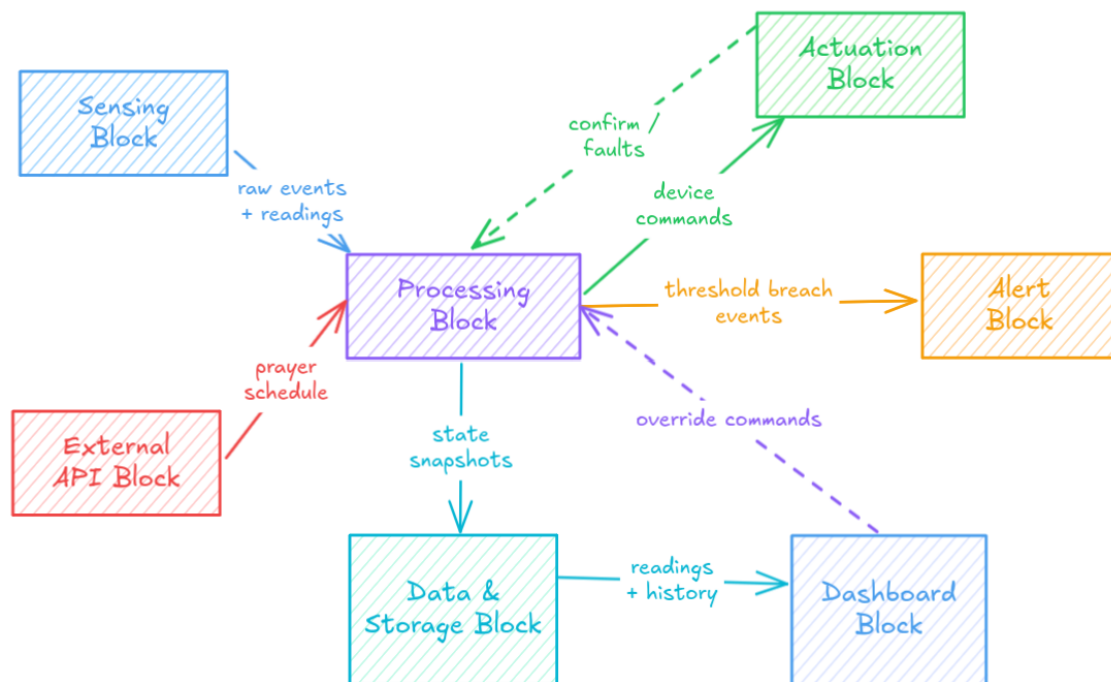


Figure 9: Inter-block data flow

10. Operational View Specification

10.1 System Performance

10.1.1 Response Time

The system operates in near real-time. Motion events trigger lighting and ventilation in approximately 1–2 seconds. In the wudu area, abnormal continuous flow triggers valve closure in approximately 2–3 seconds to reduce waste while limiting false positives.

Air quality is monitored periodically and ventilation is adjusted when thresholds are exceeded. Temperature and humidity are also tracked to maintain comfort. The system handles simultaneous sensor inputs during peak times without noticeable degradation.

The graph proves that, considering the mosque’s system operates over a local network, the typical response time for light activation or ventilation devices will not exceed 2 seconds.

The system continuously monitors CO₂ and other pollutants and responds when levels exceed safe ventilation thresholds. Sensors provide regular readings, enabling quick response to environmental changes while maintaining indoor comfort.

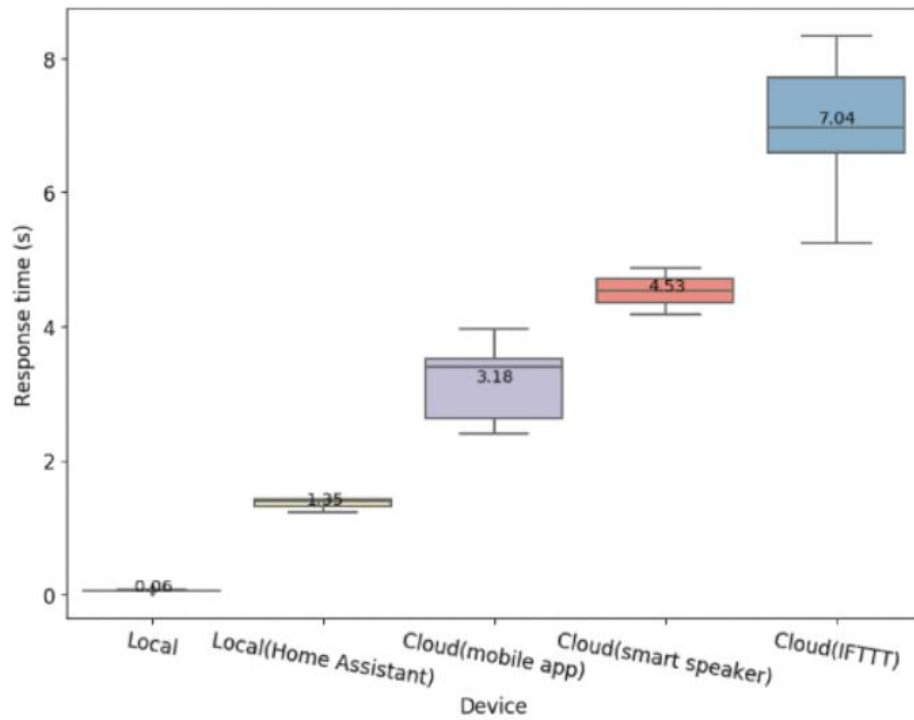


Figure 10: Operational response-time trend

10.1.2 Data Update Frequency

- Occupancy: real-time update per entry/exit event.
- Air quality: every 1 minute (balanced responsiveness and stability).
- Water usage: continuously monitored in real time.

The system updates occupancy immediately after every entry or exit event so the dashboard always reflects the current number of worshippers inside the prayer hall. This immediate response is important for crowd management during high-attendance periods.

When the mosque becomes crowded, ventilation can also be triggered proactively based on occupancy levels before the next air-quality update cycle to maintain airflow and comfort.

For water usage, the system continuously tracks total consumption and detects abnormal flow behavior such as leaks or taps left open. If abnormal flow is detected, the system can trigger alerts and automatically close the smart valve to reduce waste.

10.1.3 Throughput

The system processes motion, water-flow, and air-quality streams in parallel, enabling simultaneous monitoring and decision-making during high-demand periods such as Friday prayer and Ramadan.

The system is designed to handle multiple sensors simultaneously by collecting occupancy, water, and air-quality data in parallel and forwarding readings to the controller without blocking other operations. This ensures the dashboard remains responsive even during peak activity periods.

10.1.4 Efficiency

Lights and fans operate only when needed based on occupancy and schedule, and water control logic closes supply during leak-like behavior to reduce waste.

The system avoids unnecessary device activation by ensuring that lights and ventilation operate only when occupancy is detected and prayer-time conditions require activation. Water efficiency is also improved through continuous flow monitoring and automatic shutoff during abnormal usage.

10.2 System Reliability

10.2.1 Sensor Failure Handling

Fallback mechanisms keep operation safe during sensor faults.

- Motion sensor failure: admin manual override controls lighting and fans.
- Flow sensor failure: manual valve override + admin alert.
- Air-quality sensor failure: ventilation follows safe default schedule.

The system remains operational even if one sensor becomes unavailable, allowing maintenance to occur without interrupting mosque services.

10.2.2 Network Failure

Core control continues locally if network/server connectivity is lost. Dashboard synchronization resumes automatically after restoration.

Essential functions such as lights, fans, and water shutoff continue operating locally through the controller even during temporary internet or server outages.

10.2.3 Power Failure

On recovery, the system returns to predefined safe startup states. Critical monitoring may be maintained through optional backup power.

If backup power is available, critical monitoring functions can continue during outages. After power restoration, devices return to predefined safe states instead of restarting unpredictably.

10.3 System Scalability

10.3.1 More Sensors

Additional occupancy, flow, and IAQ sensors can be integrated with the same controller logic.

New sensors can be added gradually as mosque requirements expand. Additional occupancy sensors, flow sensors, and air-quality sensors can extend monitoring coverage without redesigning the entire system.

10.3.2 Spreading to Other Mosques

The architecture can be replicated across multiple mosques, each with local operation and data separation.

The same framework can be reused in multiple mosque deployments while maintaining independent local operation and monitoring for each location.

10.3.3 Modular Architecture

Occupancy, water, air-quality, and dashboard modules can evolve independently, reducing maintenance risk and easing upgrades.

The window-ventilation and humidity-comfort modules operate independently and were added without modifying the core occupancy or air-quality logic, demonstrating the modular design principle in practice.

This modular approach allows updates, repairs, and future feature additions to occur with minimal disruption to other system components.

10.4 System Security

10.4.1 Network Protection

- Segmented network for IoT devices
- Restricted dashboard/control access
- Firewall/gateway policy enforcement
- Monitoring for suspicious behavior and repeated login attempts

All IoT devices communicate through a secure local network rather than being openly exposed. This reduces external interference risks and improves protection for sensors and actuators.

- Sensors, controllers, and actuators operate on a separate secured network segment.
- Only approved users and services can access dashboard and device controls.
- Firewall and gateway policies reduce unauthorized access risks.
- Network traffic is monitored for suspicious activity and repeated login attempts.

10.4.2 Access Control

Role-based permissions, authenticated overrides, and action logging are applied for accountability.

Only authorized users can access the dashboard or override lighting, ventilation, and water-control systems. The administrator receives full control privileges while other users may receive limited or read-only access.

Manual overrides require authentication credentials, and all override actions are logged for accountability and auditing purposes.

Authentication flow:

User → Login Page → Authentication Check → Access Granted → Dashboard

10.4.3 Data Encryption

WPA2-PSK secures Wi-Fi communication between sensors, controllers, and dashboard. AES encryption protects traffic confidentiality and integrity.

Data transmitted across the wireless network is protected using WPA2-PSK encryption. AES encryption secures data confidentiality and helps prevent unauthorized interception or modification of occupancy, water-usage, and device-control data.

- Secures communication between sensors and controllers.
- Protects local dashboard traffic.
- Restricts network access to authorized devices only.
- Provides strong security while remaining suitable for embedded IoT devices.

10.4.4 Device Filtering

MAC allow-list filtering blocks unknown devices from joining. It is used together with WPA2-PSK and access control (not standalone security).

- Only trusted devices are allowed to join the network.
- Unknown devices are denied access.
- Suitable for small controlled IoT deployments.
- Used together with WPA2-PSK and role-based access control.
- Simplifies secure enrollment of new sensors and controllers.

10.4.5 Privacy Protection

The system stores aggregate occupancy counts only and avoids personal identification. PIR-based sensing does not capture images or personally identifiable information.

10.5 Energy Efficiency

10.5.1 Prayer Time Control

Prayer-time-aware control avoids continuous operation outside worship periods.

Prayer-time automation ensures that lighting and ventilation activate only during scheduled worship windows, reducing unnecessary energy consumption during low-use periods.

10.5.2 Occupancy-Based Activation

Devices activate only when people are present, reducing unnecessary consumption.

Occupancy detection prevents unnecessary activation when the prayer hall is empty, even if the prayer-time window is active.

10.5.3 Automatic Shutdown

Lights and fans turn off after inactivity timeout, minimizing waste.

Automatic shutdown after inactivity reduces unnecessary energy usage when worshippers leave or when only part of the mosque is occupied.

10.6 Safety Considerations

10.6.1 Device Overheating

Thermal-safe operation requires ventilation, safe power design, and alerts/shutdown under abnormal heating.

The system includes safeguards to prevent overheating of controllers, relays, and power modules during long operating periods.

10.6.2 Water Valve Reliability

Valve closure is validated over a short window to avoid false shutdown during normal use interruptions.

The water valve verifies abnormal flow conditions for a short duration before activation, reducing false shutdowns during normal ablution usage.

11. Device and Component Integration

11.1 Input Devices (Sensors)

11.1.1 Motion Sensor (PIR)

A Passive Infrared (PIR) sensor is a type of motion detection device that operates without actively emitting energy. The term “passive” indicates that the sensor does not generate or radiate signals for detection purposes. Instead, it detects variations in infrared radiation naturally emitted or reflected by objects within its field of view. PIR sensors are compact, cost-effective, and energy-efficient, making them suitable for a wide range of applications. They are commonly used in residential, medical, and industrial environments for occupancy and motion detection.



Figure 11: PIR motion sensor (left) and PIR sensing grid illustration (right)

Sensor details:

- Detection range: up to 7 meters
- Detection angle: 110 degrees
- Operating voltage: DC 4.5V–12V DC
- Output signal: 3.3V digital output
- Delay time: adjustable from 0.3 seconds to 5 minutes
- Operating temperature: -15°C to +70°C

- Sensitivity: Adjustable

Working Principle of a PIR Motion Sensor

1. Infrared detection:

A PIR sensor contains a pyroelectric element that detects changes in infrared radiation. Since all objects emit infrared energy, warmer objects produce a stronger signal.

2. Sensor structure:

The sensing element is divided into two sections that are arranged to balance each other under normal conditions. A lens is placed over the sensor to improve its sensing range and focus incoming infrared radiation.

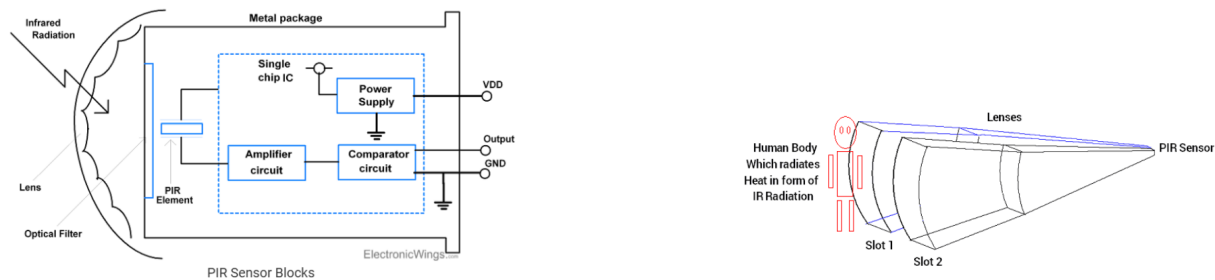


Figure 12: Infrared detection concept (left) and PIR sensor structure (right)

3. Idle state:

When no movement is present, both sections receive approximately the same amount of infrared radiation. In this case, no differential signal is produced, so the output remains stable.

4. Motion detection

When a warm body moves across the sensor's field of view, one section receives more infrared radiation than the other. This creates a differential signal, which is then amplified and compared by the internal circuit.

5. Output generation

If the change in infrared radiation is significant, the comparator produces an output signal, indicating motion. When the object leaves the sensing area, the signal changes again, allowing the sensor to detect movement in real time.

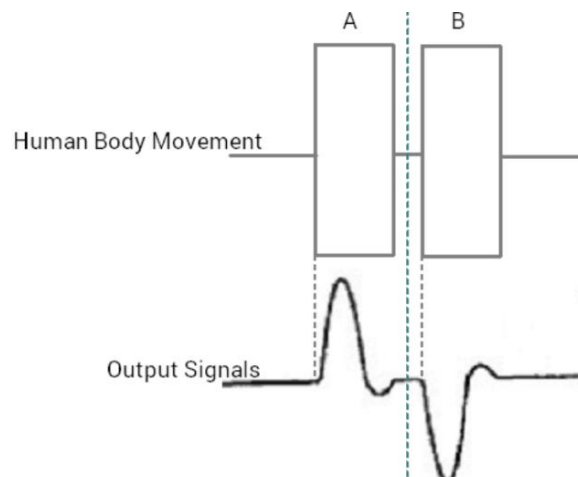


Figure 13: Motion detection behavior

It is important to remember that the PIR sensor should be installed away from the Wi-Fi antenna to avoid affecting its performance, since Wi-Fi signals emit electromagnetic radiation that can interfere with the PIR sensor.

11.1.2 Air Quality Sensor (IAQ)

To manage and maintain clean indoor air, IAQ (Indoor Air Quality) sensors are employed to monitor key environmental pollutants and climate parameters. These sensors play a central role in air purification systems, HVAC control, building automation, and environmental safety.



Figure 14: Indoor air quality sensor module

Key factors influencing IAQ:

- Chemical pollutants (CO₂, VOCs, NO₂)
- Biological contaminants (mold, bacteria, allergens)
- Particulate matter (PM2.5, PM10)
- Environmental conditions (humidity, temperature, air pressure)

The IAQ sensor is essentially a multi-parameter electronic device that detects and quantifies various pollutants and environmental conditions within indoor spaces. These sensors may measure gases, particles, and climate-related parameters, then transmit the data to a monitoring or control system. The Smart Mosque system is designed to use a multi-sensor module rather than a single-sensor one.

Common pollutants monitored by IAQ and their effects:

Working principles of IAQ sensors:

1. Sensing: Each sensor type uses a specific physical or chemical method to detect its target pollutant.
2. Signal Processing: Detected signals are converted into electrical output (analog or digital).
3. Calibration: Sensors are calibrated for accuracy, often using reference gases.
4. Data Output: The processed data is sent via I2C, UART, Modbus, or wireless protocols (Wi-Fi, Zigbee, LoRa).

Pollutant	Description	Health Effects
CO ₂	Exhaled by humans; indicator of ventilation	Fatigue, headaches, reduced productivity
VOCs	Emitted by paints, cleaners, plastics	Eye irritation, dizziness, cancer risks
PM _{2.5} /PM ₁₀	Fine dust, smoke, allergens	Respiratory and cardiovascular problems
CO	Incomplete combustion product	Fatal at high concentrations
Formaldehyde	Emitted by furniture, glues, insulation	Carcinogenic, irritation
NO ₂ /O ₃	From gas stoves, outdoor pollution	Respiratory distress
Humidity	Excessive moisture promotes mold	Discomfort, mold growth
Temperature	Affects comfort and productivity	Fatigue, poor cognitive performance

Table 18: IAQ factors and effects

5. Display and Analysis: Results are visualized on display or uploaded to the cloud for remote monitoring.

Commonly, the output of an IAQ sensor can range from 0–50 (good), 51–100 (moderate), 101–150 (unhealthy for sensitive groups), and 151–200 (unhealthy).

A good quality sensor offers accurate readings, but like any sensor, IAQ devices come with several challenges:

- Sensor drift and degradation over time
- Cross-sensitivity (e.g., VOC sensors reacting to humidity)
- Need for periodic calibration
- Data overload in multi-sensor networks
- False readings due to temperature/humidity interference

It is important to calibrate the sensor every 6–12 months to ensure good quality readings.

11.1.3 Water Flow Sensor

Water flow sensors are installed at water sources or pipes to measure the flow rate of water and calculate the total amount of water flowing through the pipe. Flow rate is commonly measured in liters per hour or cubic meters. The sensor measures water flow in real time and provides pulse output that can easily connect to an MCU in simulation or real implementation.

Working principles of a water flow sensor:

- A water flow sensor has a plastic body or valve that allows water to pass through it.



Figure 15: Water flow sensor

- Inside the sensor, a rotor and a Hall-effect sensor work together to detect and measure water flow.
- When water moves through the sensor, it rotates the rotor.
- The rotor's movement changes the rotation speed, which is converted into a pulse signal by the Hall-effect sensor.
- The pulse signal is then used to calculate the water flow rate.
- The main working principle is the Hall effect, where rotor motion induces a voltage difference measured by the sensor.
- This induced voltage represents flow movement inside the sensor.
- The flow value can then be displayed or processed by the controller.

Water flow sensors can be used with hot, cold, warm, clean, or dirty water depending on sensor type. In this project, clean water compatibility is sufficient. These sensors can be connected easily to microcontrollers such as the ESP32. The required supply voltage is typically between 5V and 18V DC.

11.2 Output Devices (Actuators)

11.2.1 Smart Lights

Smart lights are used as output devices to provide illumination in the mosque when needed. Their role is to support comfort and visibility during prayer times while reducing unnecessary energy usage during low occupancy periods.

Working principle:

- The system checks whether prayer time is active.
- It also checks whether a PIR sensor detects people in a specific area.
- If both conditions are true, lights turn ON automatically.
- If no people are detected, lights remain OFF even during the prayer window.
- After prayer time ends, lights turn OFF automatically.

How it functions in the project:

When the PIR sensor detects movement in a mosque section, the system interprets this as occupancy. If occupancy occurs during the prayer period, the lights activate automatically to provide illumination. When the area becomes empty, the lights turn OFF after a short delay to avoid unnecessary power consumption.

11.2.2 Fans (AC Simulation)

Fans and AC units maintain comfortable environmental conditions by responding to temperature, humidity, CO₂, and air-quality conditions.

Working principle:

- The IAQ sensor continuously measures temperature, humidity, and CO₂ levels.
- Ventilation fans activate when CO₂ exceeds thresholds, temperature exceeds 28°C, or humidity exceeds 65%.
- A dedicated AC unit activates when temperature exceeds 32°C as a stronger cooling response.
- If environmental conditions remain acceptable, fans and AC stay OFF or operate at lower intensity.
- When conditions improve, the system reduces ventilation automatically.

How it functions in the project:

When the mosque becomes crowded, environmental readings indicate increased ventilation demand. The controller activates fans or AC accordingly to maintain comfort and airflow. Once temperature, humidity, and CO₂ return to acceptable levels, ventilation decreases automatically to reduce energy consumption.

11.2.3 Solenoid Valve

The solenoid valve controls the water supply in the ablution area using water flow sensor readings. Its role is to stop water waste when abnormal or continuous flow is detected.

Working principle:

- Water flow sensors continuously monitor water movement.
- If abnormal flow continues longer than expected, the controller interprets this as possible waste.
- The controller sends an electrical signal to the solenoid valve.
- The solenoid valve closes and stops water flow.
- The valve can reopen manually or automatically when normal conditions return.

How it functions in the project:

If the system detects water flowing longer than expected, it assumes a tap may have been left open or a leak is present. The controller activates the solenoid valve to stop the flow and prevent unnecessary waste.

11.2.4 Smart Window

The smart window improves airflow while reducing unnecessary cooling loss.

Working principle:

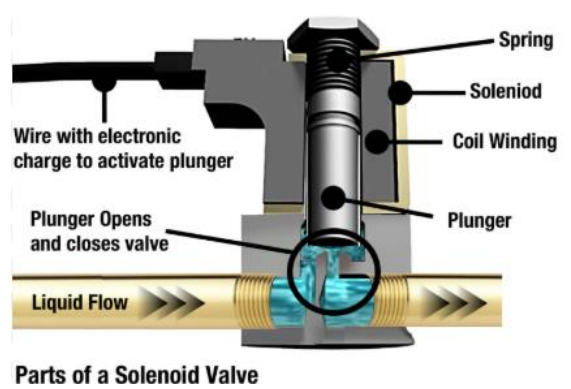


Figure 16: Solenoid valve components

- The system continuously monitors humidity and AC state.
- If the AC unit is ON, the smart window closes automatically.
- If the AC unit is OFF and humidity is high, the window opens for passive ventilation.
- Window operation updates dynamically according to environmental conditions.

How it functions in the project:

The smart window complements the ventilation system by providing natural airflow when mechanical cooling is not required, improving comfort and reducing unnecessary AC operation.

11.2.5 Humidifier

The humidifier maintains comfortable indoor humidity levels inside the mosque.

Working principle:

- Humidity sensors continuously monitor indoor moisture levels.
- If humidity falls below 30%, the humidifier activates automatically.
- Once humidity returns to acceptable range, the humidifier deactivates automatically.

How it functions in the project:

The humidifier improves comfort for worshippers during dry environmental conditions and helps maintain stable indoor air quality.

11.2.6 Street Lamp

The street lamp provides lighting near the mosque entrance during periods of activity.

Working principle:

- PIR motion sensors monitor activity near the entrance.
- When motion is detected, the controller activates the street lamp automatically.
- After inactivity, the street lamp turns OFF automatically.

How it functions in the project:

The street lamp improves entrance visibility and safety while minimizing unnecessary energy consumption during low-activity periods.

11.3 Processing Unit

The ESP32 microcontroller is selected as the processing unit because it is a powerful, low-cost, and low-power System-on-Chip (SoC) capable of supporting real-time IoT automation.

- Wi-Fi connectivity: Built-in Wi-Fi supports network-based monitoring and automation.
- Multiple interfaces: Supports GPIO, I2C, UART, SPI, and PWM interfaces.
- Low power operation: Suitable for continuous operation throughout the day.
- Embedded IoT control: Capable of handling sensor reading and device actuation simultaneously.
- Practical for project scope: Well suited for managing occupancy, ventilation, lighting, and water-control systems together.

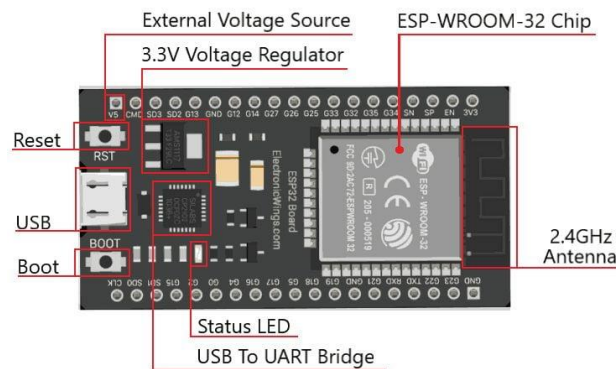


Figure 17: ESP32 processing unit

11.4 Communication Layer

The communication layer enables reliable real-time data exchange between distributed sensors, the processing unit, and the dashboard.

The system communication is implemented as follows:

- Network Setup: A local Wi-Fi access point connects ESP32 nodes to the server.
- Protocol and Security: IEEE 802.11 Wi-Fi secured using WPA2-PSK encryption.
- Topology: Star topology simplifies management and expansion.
- Network Segmentation: IoT devices operate on a separate secured network segment.
- Real-World Implementation: MQTT or Zigbee-based solutions may be used in larger deployments.
- Simulation Representation: Cisco Packet Tracer models the system using MCU nodes connected to a Registration Server.

11.5 Integration Workflow

The integration workflow follows the same process defined in the system process specification to ensure consistency between sensing, processing, and actuation stages.

1. **Sensing Stage:** Sensors continuously monitor occupancy, environmental conditions, humidity, temperature, and water usage.

2. **Data Transmission Stage:** Sensor readings are transmitted through the local Wi-Fi network to the processing unit.
3. **Processing and Decision Stage:** The controller evaluates incoming data using predefined thresholds, prayer schedules, and occupancy logic.
4. **Actuation Stage:** Control signals are sent to lighting, ventilation, windows, humidifiers, and water-control devices.
5. **Monitoring and Feedback Stage:** Dashboard systems display current sensor readings, alerts, and device status while allowing administrator overrides.

This workflow directly reflects the process flow defined earlier and ensures consistency across design and implementation.

11.6 System Diagram

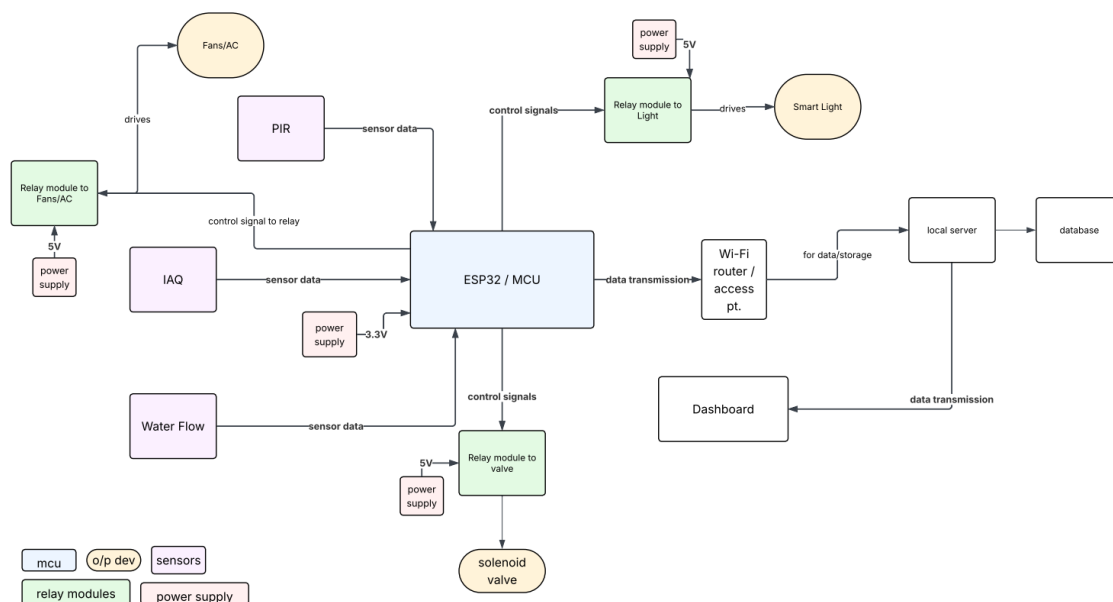


Figure 18: System integration diagram

12. Application Development

12.1 Application Overview and Platform

The application layer of the IoT system consists of a web-based dashboard that serves as the main interface between users and the system. It provides a centralized platform for interacting with real-time data collected from sensors and controlling connected devices.

Its main purpose is to enable effective monitoring and management of the IoT environment by presenting live information in a clear and organized way. The dashboard acts as the primary operational tool for users, allowing them to observe system status, respond to alerts, and manage device behavior efficiently.

The application layer is a web-based dashboard acting as the primary interface between users and the IoT system. It centralizes real-time data visualization and device control in one operational view.

12.2 User Role

The system is designed exclusively for admin users, who are responsible for monitoring and managing the IoT environment through the web-based dashboard. The admin has full access to real-time sensor data, system status, and device controls. Their role includes overseeing environmental conditions, responding to alerts, and adjusting system settings when necessary to ensure optimal performance and safety.

The system is designed for an admin role with full access to monitoring, alert handling, and override control to maintain system performance and safety.

12.3 Core Features

The web-based dashboard provides a set of essential functionalities that enable effective monitoring and control of the IoT system, including the following core features:

- Real-time monitoring of sensor data (e.g., CO₂, temperature, humidity, occupancy)
- Live system status display for all connected devices
- Alert and notification system for abnormal conditions
- Control of IoT actuators (e.g., ventilation system)
- Overview of historical data visualization and basic trends (e.g., peak hours)

The dashboard also supports monitoring of additional smart devices integrated into the system, including:

- Smart window state (open/closed)
- Humidifier activation status
- Occupancy-full siren alerts
- Street lamp activity status

The web-based dashboard communicates with the IoT system through a backend layer that enables real-time data exchange and control. It continuously receives updated sensor readings and system statuses, while also sending user commands to connected devices for execution.

Application-system interaction:

- Retrieves real-time data from IoT sensors via backend APIs
- Displays continuously updated system status on the dashboard
- Sends control commands to actuators (e.g., ventilation system)
- Supports bidirectional communication for monitoring and control
- Ensures low-latency updates for real-time system responsiveness

12.4 User Interface Design

The user interface is designed to provide a simple and organized operational dashboard that allows administrators to monitor environmental conditions and control connected IoT devices efficiently.

The interface emphasizes:

- Clear visualization of live sensor data
- Easy identification of abnormal conditions and alerts
- Simple actuator control and manual override operations
- Continuous monitoring of occupancy and environmental comfort
- Responsive layout suitable for monitoring multiple system components simultaneously

The dashboard interface includes:

- Prayer-time schedule display
- Real-time occupancy indicators
- Temperature, humidity, and CO₂ monitoring panels
- Device status indicators for lights, fans, AC, smart window, humidifier, and street lamp
- Alert notifications for overcrowding, poor air quality, abnormal water flow, and system faults

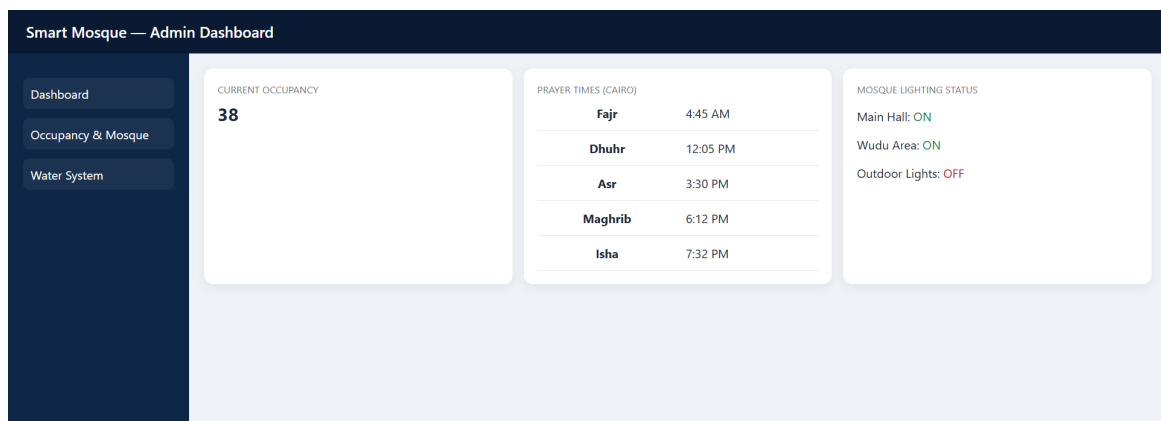


Figure 19: Dashboard UI design – Prayer times

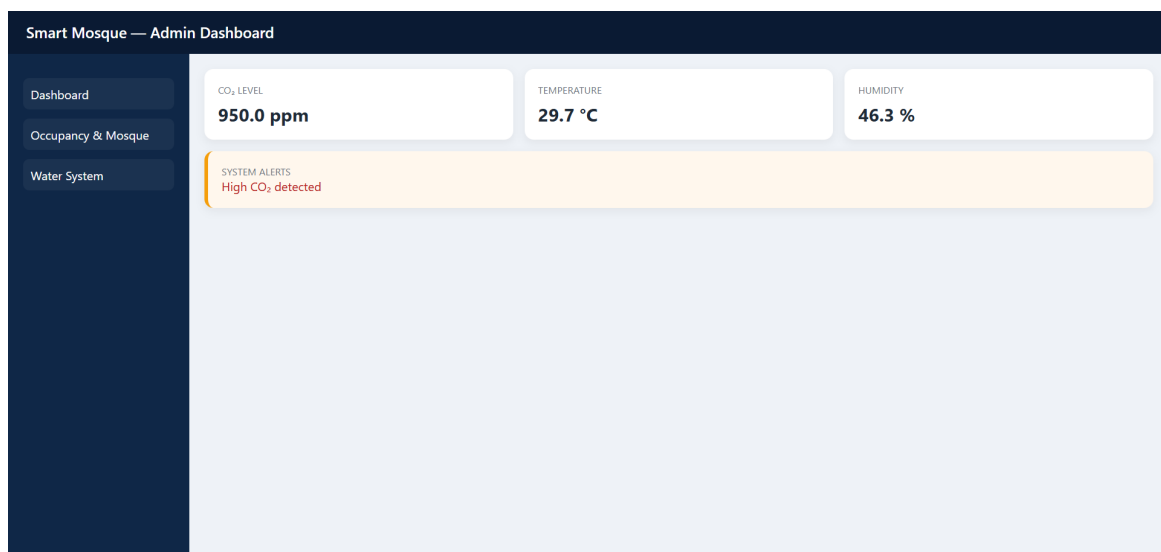


Figure 20: Dashboard UI design – High CO₂ alert

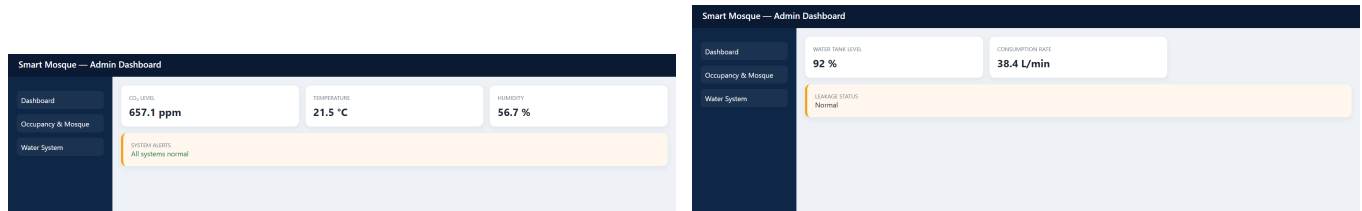


Figure 21: Dashboard UI design – Normal operating conditions

12.5 System Integration with the Dashboard

The dashboard is integrated directly with the IoT infrastructure and continuously exchanges information with sensors, controllers, and actuators through the local network.

The integration workflow includes:

- Receiving live sensor readings from occupancy, IAQ, humidity, and water-monitoring systems
- Processing and displaying environmental thresholds and alerts
- Sending manual override commands to lighting, ventilation, humidifier, smart window, and valve-control devices
- Maintaining synchronized device-state visualization for administrators
- Supporting reliable low-latency monitoring inside the local mosque network

List of Figures

1	Integrated system flowchart	8
2	UML domain model diagram	11
3	UML domain model diagram	14
4	Main data flow	16
5	Main flow legend	16
6	Control flow	16
7	Energy control logic finite state machine	18
8	IoT system layer interactions	27
9	Inter-block data flow	29
10	Operational response-time trend	30
11	PIR motion sensor (left) and PIR sensing grid illustration (right)	34
12	Infrared detection concept (left) and PIR sensor structure (right)	35
13	Motion detection behavior	35
14	Indoor air quality sensor module	36
15	Water flow sensor	38
16	Solenoid valve components	40
17	ESP32 processing unit	41
18	System integration diagram	42
19	Dashboard UI design – Prayer times	44
20	Dashboard UI design – High CO ₂ alert	44
21	Dashboard UI design – Normal operating conditions	45

References

- PIR Motion Sensor: <https://www.electronicwings.com/sensors-modules/pir-sensor>
- IAQ Sensor: <https://www.winsen-sensor.com/knowledge/iaq-sensor.html>
- Water Flow Sensor: <https://www.elprocus.com/a-memoir-on-water-flow-sensor/>
- ESP32 Processing Unit: <https://www.espressif.com/en/products/socs/esp32>
- IoT Response Time: <https://ieeexplore.ieee.org/document/9043102>
- Carbon Dioxide Levels:
<https://www.co2meter.com/blogs/news/carbon-dioxide-indoor-levels-chart>
- Air Quality Monitoring:
<https://www.sciencedirect.com/science/article/pii/S0360132325008686>
- Water Flow Monitoring: <https://new.abb.com/products/measurement-products/measurement-products-blog/water-flowmeters-for-enhanced-leakage-detection>
- IoT Security: <https://www.digi.com/blog/post/iot-security>
- Access Control: <https://thingsboard.io/docs/pe/user-guide/rbac/>
- Data Encryption:
<https://www.buildings.com/safety-security/cybersecurity/article/33017912/trust-no-one-3-steps-to-secure-iot-devices-and-building-it-and-ot-networks>
- WPA2-PSK Background: <https://superops.com/tech-hub/what-is-wpa2-psk>
- Device Filtering: <https://www.portnox.com/cybersecurity-101/network-security/what-is-a-mac-filtering-router/>
- MAC Filtering: <https://blog.scalefusion.com/what-is-mac-filtering/>